

MODUL 1 INPUT OUTPUT DIGITAL DAN INTERUPSI EKSTERNAL

Muhammad Aqeel Ghani (13224071)

Asisten: Nisrina Azzah Atiqoh

Tanggal Percobaan: 24/09/2026

EL3112-Praktikum Sistem Mikroprosesor

Laboratorium Dasar Teknik Elektro - Sekolah Teknik Elektro dan Informatika ITB



Abstrak

Praktikum ini bertujuan untuk menganalisis karakteristik kinerja sistem input/output (I/O) digital dan mekanisme interupsi eksternal pada mikrokontroler ESP32. Pengujian dilakukan melalui tujuh rangkaian percobaan, meliputi pengukuran identitas hardware dan tegangan catu daya, perbandingan kecepatan eksekusi dan latensi I/O antara metode Arduino Library dan Direct Register Access, karakteristik waktu respon sakelar relay elektromekanik, serta batas frekuensi pencacahan pulsa berbasis external interrupt. Hasil percobaan menunjukkan bahwa pemodelan catu daya ESP32 menghasilkan tegangan konstan sebesar 3,223 V dengan alamat MAC d8:bc:38:f7:d3:54. Penggunaan metode Direct Register Access terbukti secara signifikan meningkatkan kecepatan toggling rate output digital hingga mencapai 4,000 MHz (meningkat 208% dibandingkan Arduino Library yang sebesar 1,299 MHz pada clock CPU 240 MHz) serta menghasilkan sinyal persegi simetris dengan duty cycle 50,0%. Pada pengujian latensi I/O, akses register mampu menurunkan latensi hingga 486,66 ns pada frekuensi 100 kHz dan memperluas batas frekuensi masukan hingga 1 MHz sebelum mengalami pergeseran fasa. Pengujian aktuator relay mengindikasikan adanya penundaan respon mekanis sebesar 3,78 ms (rising edge) dan 3,50 ms (falling edge). Sementara itu, pengukuran frekuensi menggunakan interupsi eksternal memiliki batas akurasi hingga frekuensi 200 kHz, di mana peningkatan frekuensi ke 456 kHz menyebabkan fenomena lost pulses akibat overhead eksekusi ISR dan context switching CPU.

Kata kunci: ESP32, Direct Register Access, Latensi I/O, Relay, Interupsi Eksternal.

1. PENDAHULUAN

Laporan ini disusun sebagai hasil Praktikum Modul 1 Matakuliah Praktikum Sistem Mikroprosesor yang berjudul Input Output Digital dan Interupsi Eksternal. Modul 1 Praktikum Sistem Mikroprosesor menjalankan tujuh percobaan yaitu :

- 1) Persiapan (Mencatat Identitas Modul ESP32 dan Mengukur Tegangan VCC pada ESP32)
- 2) Mengukur Kecepatan Output Digital dengan Library Arduino

- 3) Mengukur Kecepatan Output Digital dengan Akses Register
- 4) Latency Input Output Digital dengan Library Arduino
- 5) Latency Input Output Digital dengan Akses Register
- 6) Pengendali Relay
- 7) Pengukur Frekuensi

Modul ini memiliki tiga tujuan utama yaitu :

- 1) Membuat aplikasi dengan Visual Code dan Arduino IDE
- 2) Membuat aplikasi input dan output digital pada ESP32 dengan menggunakan bahasa pemrograman C
- 3) Menggunakan interupsi eksternal untuk input digital

2. STUDI PUSTAKA

2.1 ARSITEKTUR GPIO ESP32 DAN METODE AKSES REGISTER

ESP32 merupakan 2.4 GHz Wi-Fi dan Bluetooth combo chip yang didesain dengan teknologi low-power 40 nm TSMC. ESP32 memiliki tegangan kerja sebesar 3.3 Volt dengan Absolute Maximum Rating sebesar 3.6 Volt [1]. Pengendalian I/O Digital dapat dilakukan melalui dua metode utama, yaitu Direct Register Access dan Arduino Library. Pengendalian I/O menggunakan metode Direct Register Access bekerja dengan menuliskan langsung menuliskannya di memory-mapped register. Dalam ESP32, terdapat IO MUX, RTC IO MUX, dan GPIO Matrix yang bekerja mengurus routing ke pin nya [2]. Sedangkan untuk fungsi library seperti digitalWrite() harus melalui overhead terlebih dahulu untuk mensetup koneksi dengan GPIO nya. Setidaknya fungsi library harus mengakses memory untuk mendapat pin mapping, mengakomodasi kemungkinan MCU State, dan testing conditional statement. Hal ini menyebabkan fungsi library membutuhkan lebih

banyak clock cycle dibandingkan direct port manipulation [3].

2.2. Hubungan Frekuensi Clock CPU terhadap Kecepatan Output Digital

Kecepatan eksekusi instruksi pada mikroprosesor dipengaruhi secara langsung oleh frekuensi clock sistem [4]. Menurut Patterson dan Hennessy [5], waktu yang dibutuhkan oleh CPU untuk menyelesaikan eksekusi sekumpulan instruksi (T_{exec}) berbanding terbalik dengan frekuensi clock CPU (f_{CPU}), yang dinyatakan dalam persamaan berikut :

$$T_{exec} = \frac{N \times CPI}{f_{CPU}}$$

dengan N menyatakan jumlah instruksi dan CPI menyatakan rata-rata siklus clock per instruksi [5]. Pada mikrokontroler ESP32, frekuensi clock CPU diatur melalui sistem Reset and Clock Controller yang memanfaatkan sumber gelombang Phase-Locked Loop (PLL) [2]. Ketika mikrokontroler mengeksekusi siklus pembentukan gelombang persegi pada pin output digital tanpa delay, frekuensi sinyal keluaran maksimum (f_{out}) ditentukan oleh jumlah siklus clock yang dibutuhkan untuk membalikkan logika pin HIGH dan LOW:

$$f_{out} = \frac{f_{CPU}}{2 \times N_{cycles}}$$

Oleh karena itu, penurunan frekuensi clock CPU (misalnya dari 240 MHz menjadi 80 MHz) akan memperpanjang waktu eksekusi instruksi T_{exec} , sehingga menghasilkan penurunan frekuensi sinyal output digital secara linear [2].

2.3. Latency I/O Digital dan Evaluasi Statistik

Latency pada sistem I/O digital didefinisikan sebagai interval waktu yang dibutuhkan oleh sistem mikrokontroler untuk memberikan respon keluaran pada pin output setelah terjadinya perubahan logika pada pin input ($t_{latency} = t_{out} - t_{in}$). Latensi ini dipengaruhi oleh dua faktor utama, yaitu latensi perangkat keras (seperti propagation delay pada bus perifer IO MUX) dan latensi perangkat lunak (seperti durasi pembacaan instruksi program) [6]. Penggunaan high level library function cenderung menambah latensi eksekusi karena adanya pemeriksaan kondisi internal, sedangkan akses register langsung meminimalisasi siklus instruksi CPU [2]. Karena pemrosesan internal mikrokontroler dan noise pengukuran dapat bervariasi pada setiap siklus

eksekusi, respon waktu terukur bersifat stokastik [6], [7].

2.4. Rangkaian Driver Relay Elektromekanis dan Response Time

Relay elektromekanis merupakan aktuatur berbasis saklar mekanis yang dikendalikan oleh medan elektromagnetik kumparan [8]. Karena beban kumparan relay membutuhkan arus yang lebih besar daripada batas toleransi pin GPIO ESP32 serta beroperasi pada tegangan 5V, dibutuhkan rangkaian driver saklar elektronik berbasis transistor NPN atau BJT [9]. Ketika sinyal logika HIGH (3.3 V) diberikan dari pin GPIO ke basis transistor, transistor terbias ke daerah saturasi dan mengalirkan arus ke kumparan relay [9]. Untuk melindungi transistor dari lonjakan tegangan balik induktif (back-EMF) $V = L \frac{di}{dt}$ saat transistor berpindah ke kondisi mati (cutoff), dipasang flyback diode secara paralel sejajar dengan kumparan relay [9]. Kecepatan saklar relay ditentukan oleh total waktu respon (response time), yaitu jeda waktu dari pengiriman sinyal logika kontrol hingga kontak mekanis relay benar-benar terhubung stabil [8]. Response time relay mencakup penundaan magnetisasi kumparan (t_{coil}), waktu gerak fisik jangkar saklar ($t_{mechanical}$), serta fenomena pantulan kontak (contact bounce) [4], [8]:

$$t_{response} = t_{coil} + t_{mechanical} + t_{bounce}$$

Berbeda dengan kecepatan switching transistor semikonduktor murni yang berada pada orde nanodetik (ns), response time relay elektromekanis jauh lebih lambat dan berada pada orde milidetik (ms) akibat keterbatasan karakteristik mekanis komponen [4], [8].

2.5. Interupsi Eksternal dan Batas Frekuensi Pengukuran

Interupsi eksternal merupakan mekanisme asynchronous yang memungkinkan perangkat keras luar memicu penghentian sementara eksekusi program utama guna menjalankan fungsi penanganan khusus yang disebut Interrupt Service Routine (ISR) [10]. Pada ESP32, sinyal transisi logika pada pin GPIO (rising, falling, atau change) dialokasikan melalui Interrupt Matrix menuju salah satu CPU core [2]. Penggunaan interupsi eksternal jauh lebih efisien dibandingkan metode polling karena CPU tidak perlu menghabiskan siklus clock untuk mengecek status pin secara terus-menerus [10].

Meskipun demikian, pembacaan frekuensi sinyal masukan menggunakan interupsi eksternal

memiliki batasan frekuensi maksimum (f_{maks}) [4]. Ketika interupsi dipicu, CPU membutuhkan waktu untuk melakukan context switching (yaitu menyimpan register kerja CPU ke *stack*) ditambah dengan durasi eksekusi instruksi di dalam ISR [4], [6]. Total waktu penanganan interupsi ini (t_{total}) dirumuskan sebagai:

$$t_{total} = t_{latency} + t_{ISR} + t_{restore}$$

Jika periode sinyal masukan ($T_{in} = 1/f_{in}$) lebih pendek daripada t_{total} , interupsi baru akan terjadi sebelum ISR sebelumnya selesai dieksekusi [2]. Fenomena ini menyebabkan terjadinya missed pulse atau system crash akibat akumulasi penumpukan stack interupsi [4], [6]. Oleh karena itu, frekuensi batas maksimum masukan yang dapat dihitung secara akurat dibatasi oleh [4]:

$$f_{maks} \leq \frac{1}{t_{total}}$$

3. METODOLOGI

Pada Praktikum Modul 1, ada beberapa alat yang digunakan sebagai berikut :

- 1) Kit praktikum ESP32
- 2) Osiloskop
- 3) Frequency counter
- 4) Signal generator

Berikut juga merupakan langkah-langkah percobaan :



Gambar 3-1 Langkah Percobaan 1



Gambar 3-2 Langkah Percobaan 2



Gambar 3-3 Langkah Percobaan 3



Gambar 3-4 Langkah Percobaan 4



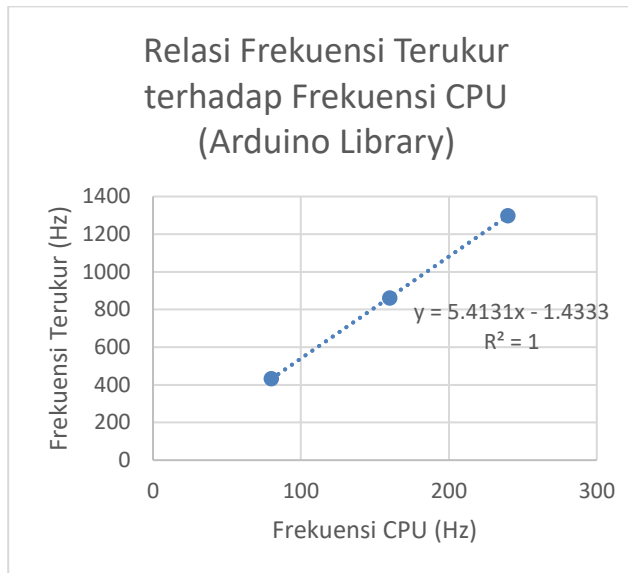
Gambar 3-5 Langkah Percobaan 5



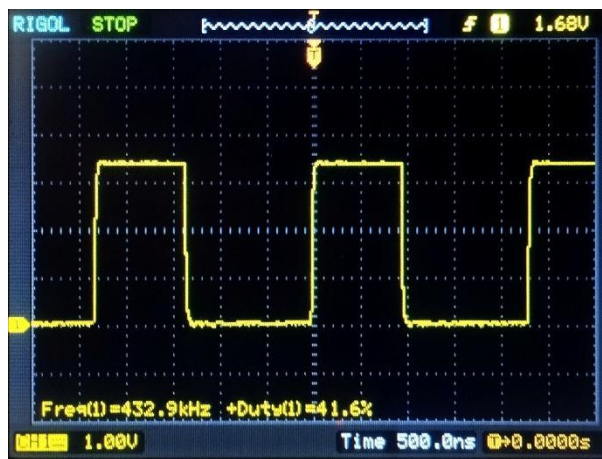
Gambar 3-6 Langkah Percobaan 6



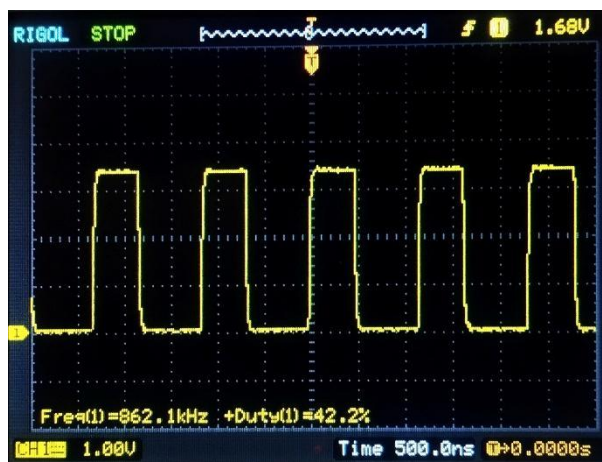
Gambar 3-7 Langkah Percobaan 7



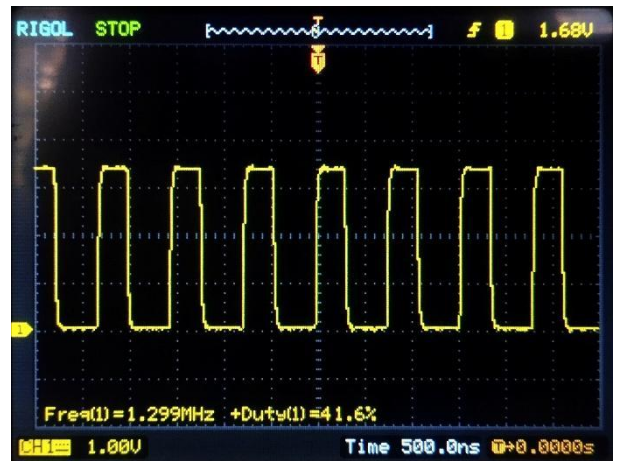
Gambar 4-2-2 Regresi Percobaan 2



Gambar 4-2-3 Osiloskop Percobaan 2 (80 MHz)



Gambar 4-2-4 Osiloskop Percobaan 2 (160 MHz)



Gambar 4-2-5 Osiloskop Percobaan 2 (240 MHz)

4.2.3. Analisis Hasil Percobaan dan Kesesuaian Teori

Berdasarkan bentuk sinyal persegi yang ditampilkan pada osiloskop, gelombang keluaran yang dihasilkan menggunakan Arduino Library `digitalWrite()` tidak simetris sempurna. Hal ini ditunjukkan oleh nilai duty cycle terukur yang berada pada kisaran 41,6% hingga 42,2% (jauh di bawah nilai simetris ideal 50%). Ketidaksimetrisan ini terjadi karena fungsi `digitalWrite()` merupakan high-level library yang memiliki overhead eksekusi cukup besar, seperti proses pemetaan nomor pin, pemeriksaan kondisi validasi register, dan konversi status logika. Selain itu, pada akhir fungsi `loop()`, terdapat overhead tersembunyi berupa pemanggilan balik fungsi `loop()` oleh sistem operasi (FreeRTOS task scheduler pada ESP32) sebelum fungsi `digitalWrite(LED_PIN, HIGH)` dieksekusi kembali. Waktu jeda tambahan ini menyebabkan kondisi logika LOW bertahan sedikit lebih lama dibandingkan logika HIGH, sehingga menurunkan persentase duty cycle sinyal.

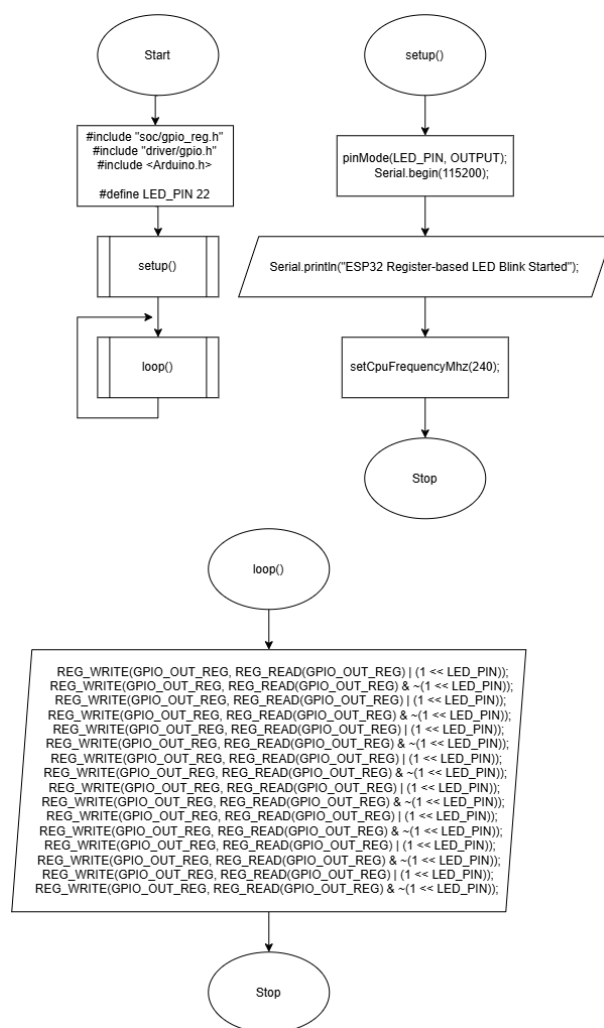
Pengaruh frekuensi clock CPU terhadap frekuensi sinyal terukur menunjukkan hubungan berbanding lurus (proporsional linier). Ketika frekuensi CPU ditingkatkan berturut-turut dari 80 MHz, 160 MHz, hingga 240 MHz, frekuensi sinyal output yang dihasilkan ikut meningkat secara signifikan dari 432,9 kHz, 862,1 kHz, hingga 1299 kHz (1,299 MHz). Hal ini sesuai dengan teori arsitektur mikrokontroler di mana durasi satu siklus instruksi (T_{clock}) berbanding terbalik dengan frekuensi clock ($T_{clock} = 1/f_{CPU}$). Semakin tinggi frekuensi clock CPU, semakin singkat waktu yang dibutuhkan prosesor untuk menyelesaikan baris instruksi `digitalWrite()`, sehingga frekuensi pergantian logika (toggling rate) menjadi lebih cepat.

Analisis regresi linier menghasilkan persamaan $\hat{y} = 5,4131x - 1,4333$ dengan nilai $R^2 = 1$, yang

membuktikan adanya korelasi positif sempurna antara peningkatan frekuensi CPU dan frekuensi sinyal keluaran. Dari segi bentuk sinyal, gelombang tetap mempertahankan karakteristik gelombang persegi (square wave) pada semua tingkat frekuensi clock. Namun, pada frekuensi terukur tertinggi (1,299 MHz pada CPU 240 MHz), efek kapasitansi parasitik pada jalur PCB dan probe osiloskop mulai menyebabkan sedikit kelengkungan (slew rate limitation) pada transisi naik (rise time) dan transisi turun (fall time) sinyal, meskipun bentuk ideal gelombang digital masih dapat teramati dengan jelas.

4.3. Mengukur Kecepatan Output Digital dengan Akses Register

4.3.1. Flowchart

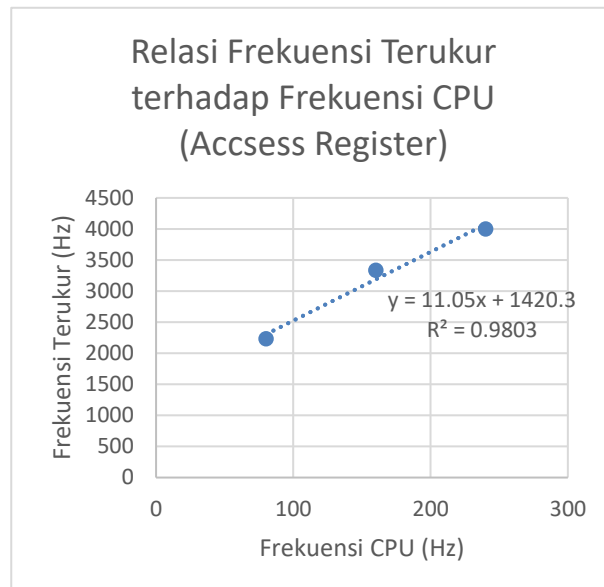


Gambar 4-3-1 Flowchart Source Code Percobaan 3

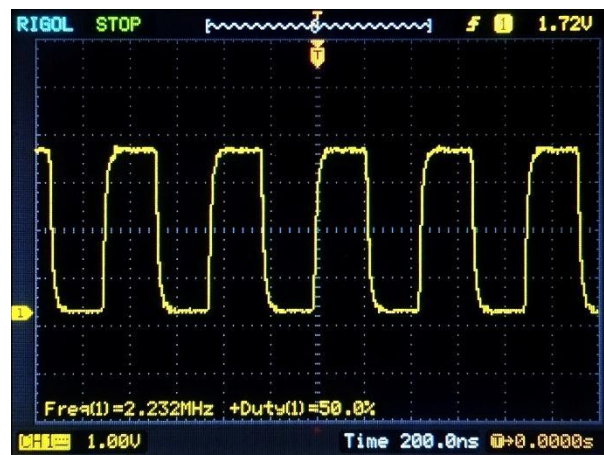
4.3.2. Data Hasil Percobaan

Tabel 4-3-1 Data Hasil Percobaan 3

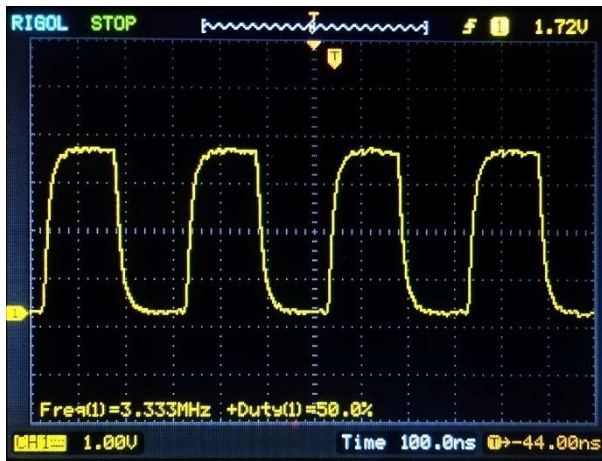
Frekuensi CPU (Hz)	Frekuensi Terukur (Hz)	Duty Cycle (%)
80	2232	50
160	3333	50
240	4000	49.6



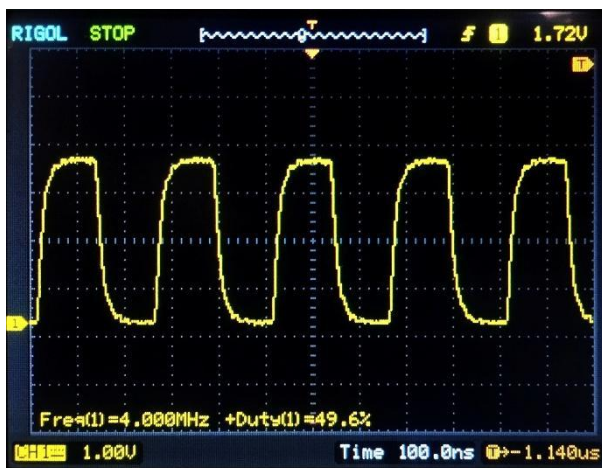
Gambar 4-3-2 Regresi Percobaan 3



Gambar 4-3-3 Osiloskop Percobaan 3 (80 MHz)



Gambar 4-3-4 Osiloskop Percobaan 3 (160 MHz)



Gambar 4-3-5 Osiloskop Percobaan 3 (240 MHz)

4.3.3. Analisis Hasil Percobaan dan Kesesuaian Teori

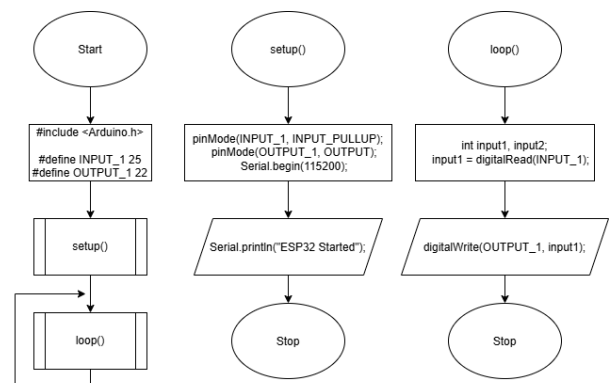
Berdasarkan hasil pengujian menggunakan metode Akses Register, sinyal keluaran yang dihasilkan pada pin GPIO 22 menunjukkan bentuk gelombang yang sangat simetris dibandingkan dengan penggunaan Arduino Library. Hal ini dibuktikan oleh nilai duty cycle terukur yang mencapai 50,0% pada frekuensi CPU 80 MHz dan 160 MHz, serta 49,6% pada frekuensi CPU 240 MHz. Simetrisitas yang tinggi ini dapat dicapai karena penulisan status logika HIGH dan LOW dilakukan secara berurutan (loop unrolling) langsung ke register memori GPIO_OUT_REG. Langkah ini berhasil mengeliminasi overhead pemeriksaan kondisi pin, pemetaan fungsi, serta waktu penundaan akibat pemanggilan balik FreeRTOS task scheduler yang sebelumnya menghambat kestabilan pulsa logika. Dari segi kecepatan eksekusi, metode akses register terbukti secara teoritis maupun praktis jauh lebih unggul. Pada frekuensi CPU maksimum 240 MHz, metode akses register mampu menghasilkan frekuensi sinyal hingga 4,000 MHz, meningkat lebih dari tiga kali lipat (sekitar 208%)

dibandingkan library digitalWrite() yang hanya mencapai 1,299 MHz.

Peningkatan frekuensi clock CPU berbanding lurus dengan frekuensi sinyal terukur, di mana peningkatan frekuensi CPU dari 80 MHz, 160 MHz, hingga 240 MHz menghasilkan frekuensi keluaran berturut-turut sebesar 2,232 MHz, 3,333 MHz, dan 4,000 MHz. Hubungan ini didukung oleh analisis regresi linier dengan persamaan $\hat{y} = 11,05x + 1420,3$ dan nilai koefisien determinasi $R^2 = 0,9803$, yang mengindikasikan adanya korelasi linier positif yang sangat kuat. Kenaikan frekuensi terukur yang sedikit melandai pada CPU 240 MHz disebabkan oleh limitasi kecepatan bus perifer APB (Advanced Peripheral Bus) internal ESP32 serta batas fisik slew rate perpindahan transistor I/O. Selain itu, dari segi bentuk sinyal pada frekuensi tinggi (3,333 MHz dan 4,000 MHz), tepi gelombang mengalami sedikit kelengkungan (rounding edge) pada transisi naik dan turun. Fenomena ini merupakan dampak dari kapasitansi parasitik pada jalur PCB dan probe osiloskop yang mulai membatasi kecepatan respon tegangan saat logika berlangsung dalam orde Megahertz.

4.4. Latency Input Output Digital dengan Library Arduino

4.4.1. Flowchart



Gambar 4-4-1 Flowchart Source Code Percobaan 4

4.4.2. Data Hasil Percobaan

Tabel 4-4-1 Data Hasil Percobaan 4

Latensi Pulsa ke-	Frekuensi (kHz)		
	1	10	100
1	<40 nS	602 nS	1.1 uS
2	<40 nS	404 nS	1.116 uS
3	<40 nS	994 nS	1.126 uS
Latensi Rata-Rata	<40 nS	666 nS	1.114 uS



Gambar 4-4-2 Osiloskop Percobaan 4 saat Frekuensi melebihi batas (800 kHz)

4.4.3. Analisis Hasil Percobaan dan Kesesuaian Teori

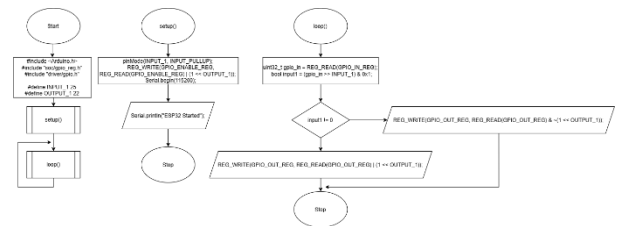
Berdasarkan data percobaan, nilai latensi antara sinyal masuk pada GPIO 25 dan sinyal keluar pada GPIO 22 menunjukkan kecenderungan meningkat seiring dengan bertambahnya frekuensi sinyal input. Pada frekuensi rendah (1 kHz), latensi terukur sangat kecil yaitu di bawah 40 ns. Namun, saat frekuensi ditingkatkan menjadi 10 kHz dan 100 kHz, nilai rata-rata latensi naik signifikan berturut-turut menjadi 666 ns dan 1,114 μ s. Fluktuasi nilai latensi antar-sampel pulsa pada frekuensi 10 kHz (berkisar antara 404 ns hingga 994 ns) mempertegas sifat latensi sistem yang bersifat statistik. Sifat non-deterministik ini disebabkan oleh overhead eksekusi fungsi `digitalRead()` dan `digitalWrite()` pada Arduino Library yang memerlukan beberapa siklus instruksi CPU untuk pengecekan validasi pin, serta adanya proses latar belakang (background task) dari sistem operasi real-time (FreeRTOS) pada ESP32.

Ketika frekuensi sinyal masukan dinaikkan hingga melebihi batas ambang respon perangkat lunak (> 800 kHz), siklus eksekusi fungsi `loop()` tidak lagi cukup cepat untuk mendeteksi setiap transisi logika masukan. Tampilan osiloskop menunjukkan pergeseran fasa yang sangat besar di mana tanggapan keluaran tertinggal jauh di belakang sinyal masukan dengan selisih waktu terukur sebesar -620,0 ns. Pada kondisi ekstrem melebihi batas ini, terjadi efek aliasing atau hilangnya pulsa (dropped pulses), di mana sinyal masukan berubah kembali sebelum program sempat menyelesaikan pembacaan dan penulisan status logika. Hal ini membuktikan bahwa batas frekuensi sampling maksimum untuk pembacaan I/O digital berbasis Arduino Library pada ESP32 berada di bawah 800 kHz. Untuk aplikasi yang membutuhkan latensi ultra-rendah dan frekuensi tinggi, diperlukan

penggunaan metode Direct Register Access atau penanganan berbasis Hardware Interrupt (ISR).

4.5. Latency Input Output Digital dengan Akses Register

4.5.1. Flowchart

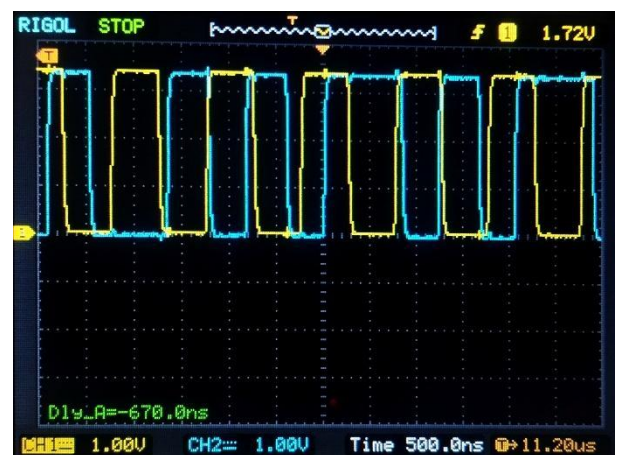


Gambar 4-5-1 Flowchart Source Code Percobaan 5

4.5.2. Data Hasil Percobaan

Tabel 4-5-1 Data Hasil Percobaan 5

Latensi Pulsa ke-	Frekuensi (kHz)		
	1	10	100
1	<40 nS	606 nS	584 nS
2	<40 nS	518 nS	488 nS
3	<40 nS	604 nS	388 nS
Latensi Rata-Rata	<40 nS	576 nS	486.66 nS



Gambar 4-5-2 Osiloskop Percobaan 5 saat Frekuensi melebihi batas (1 MHz)

4.5.3. Analisis Hasil Percobaan dan Kesesuaian Teori

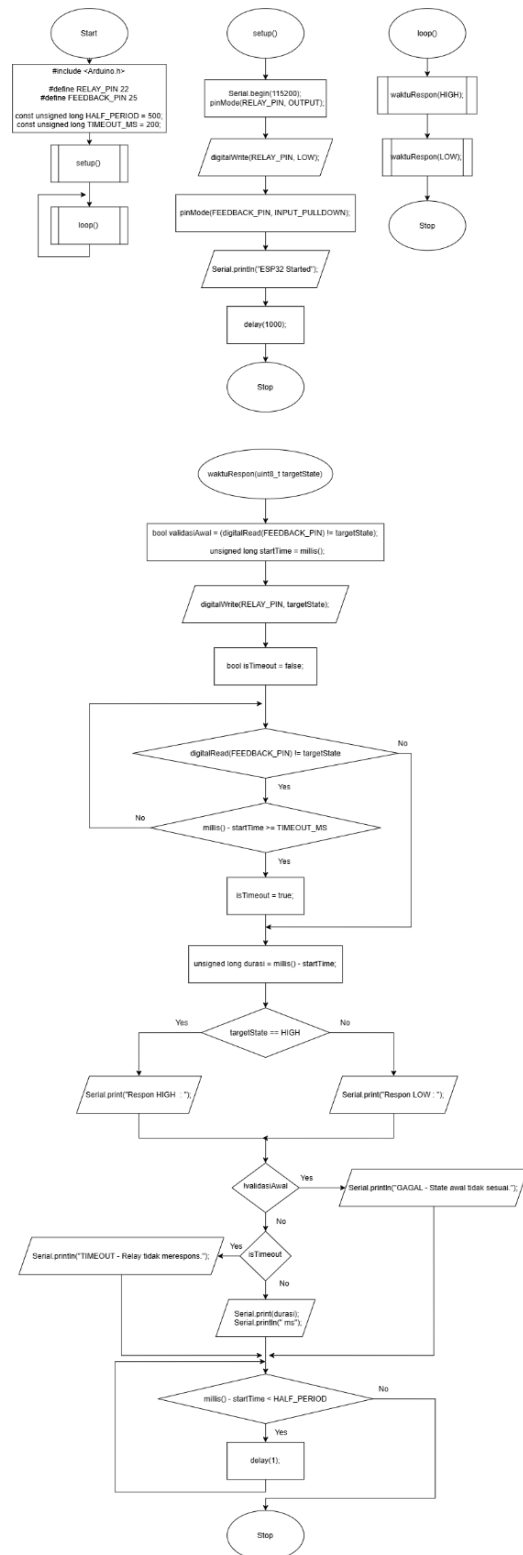
Berdasarkan data percobaan, penggunaan metode Direct Register Access terbukti secara signifikan menurunkan latensi respon I/O digital jika dibandingkan dengan metode Standard Library Arduino pada Percobaan 4.1.4. Pada frekuensi 1 kHz, kedua metode memberikan respon cepat dengan latensi terukur di bawah 40 ns. Namun, perbedaan performa terlihat sangat jelas pada frekuensi yang lebih tinggi. Pada frekuensi 10 kHz, rata-rata latensi akses register turun menjadi 576 ns (dibandingkan 666 ns pada Arduino Library).

Penghematan waktu tanggapan menjadi sangat signifikan pada frekuensi 100 kHz, di mana latensi rata-rata akses register hanya sebesar 486,66 ns, sedangkan Arduino Library membutuhkan waktu hingga 1,114 μ s (1114 ns). Penurunan latensi lebih dari 56% ini terjadi karena eksekusi instruksi tingkat rendah (low-level) langsung memanipulasi memori register GPIO_IN_REG dan GPIO_OUT_REG tanpa melewati overhead pengecekan keamanan pin, mapping, dan pemanggilan fungsi internal Arduino.

Selain menghasilkan latensi yang lebih rendah, batas frekuensi masukan maksimum yang dapat ditangani oleh sistem juga meningkat. Jika Arduino Library mulai kehilangan kemampuan pembacaan dan mengalami keterlambatan signifikan pada frekuensi batas 800 kHz, metode akses register mampu merespon sinyal masukan hingga frekuensi 1 MHz sebelum mencapai batas limitasinya. Ketika frekuensi masukan dinaikkan melebihi 1 MHz, kecepatan perulangan fungsi loop() dan struktur percabangan if-else dalam kode program tidak lagi mampu mengimbangi laju transisi sinyal masukan. Tampilan osiloskop menunjukkan pergeseran fasa dengan nilai latensi sebesar -670,0 ns, yang mengindikasikan bahwa pulsa keluaran baru merespon setelah gelombang masukan berikutnya bergeser. Hal ini membuktikan bahwa pembacaan register secara langsung memperluas bandwidth tanggapan I/O sistem, meskipun batas absolut tetap dibatasi oleh arsitektur eksekusi perangkat lunak berbasis loop.

4.6. Pengendali Relay

4.6.1. Flowchart



Gambar 4-6-1 Flowchart Source Code Percobaan 6

4.6.2. Data Hasil Percobaan

Tabel 4-6-1 Data Hasil Percobaan 6

Metode Pengukuran	Osiloskop		Software	
	Rising Edge	Falling Edge	Rising Edge	Falling Edge
Waktu Respons (ms)	3.78	3.5	4	4



Gambar 4-6-2 Osiloskop Percobaan 5 saat Rising Edge



Gambar 4-6-3 Osiloskop Percobaan 5 saat Falling Edge

4.6.3. Analisis Hasil Percobaan dan Kesesuaian Teori

Berdasarkan data hasil percobaan, relay elektromekanik memiliki latensi pemicuan dalam orde milisekon (ms). Hal ini berbeda signifikan dengan kecepatan sakelar elektronik berbasis semikonduktor yang memiliki latensi dalam orde mikrodetik hingga nanodetik. Keterlambatan waktu ini disebabkan oleh mekanisme kerja relay yang bersifat mekanik: ketika sinyal kendali diberikan, kumparan (coil) membutuhkan waktu untuk membangun medan magnet yang cukup kuat untuk menarik lidah kontak fisik, begitupun

saat deactivated di mana gaya pegas membalikkan posisi kontak setelah medan magnet meluruh.

Dari pengujian osiloskop, waktu respon Rising Edge terukur sebesar 3,78 ms, sedangkan Falling Edge terukur sedikit lebih cepat yaitu 3,50 ms. Perbedaan durasi ini terjadi karena pada transisi Rising Edge, sistem harus mengatasi inersia mekanis lidah kontak dan resistansi pegas pembalik. Sementara itu, pada transisi Falling Edge, pemutusan arus pada kumparan dibantu oleh flyback diode dan dorongan pegas pembalik yang dengan cepat menarik kontak ke posisi semula.

Pengukuran berbasis software menggunakan fungsi millis() menghasilkan nilai pembulatan sebesar 4 ms baik untuk Rising Edge maupun Falling Edge. Nilai ini konsisten dan valid terhadap hasil osiloskop. Keterbatasan fungsi millis() yang memiliki resolusi terendah 1 ms menyebabkan nilai presisi osiloskop (3,78 ms dan 3,50 ms) terbilang ke atas menjadi nilai integer terdekat (4 ms). Secara keseluruhan, relay sangat cocok digunakan untuk aplikasi sakelar daya tinggi yang tidak membutuhkan kecepatan tinggi, namun tidak dapat digunakan untuk aplikasi switching frekuensi tinggi seperti PWM (Pulse Width Modulation) karena keterbatasan respon mekanis tersebut.

4.7. Pengukur Frekuensi

4.7.1. Flowchart



Gambar 4-7-1 Flowchart Source Code Percobaan 7

4.7.2. Data Hasil Percobaan

Tabel 4-7-1 Data Hasil Percobaan 7

Frekuensi (kHz)	
Generator Sinyal	Terukur
1	1
10	10
100	99.993
200	199.5
456	385

4.7.3. Analisis Hasil Percobaan dan Kesesuaian Teori

Berdasarkan data percobaan pada Tabel 4-7-1, metode pencacahan pulsa berbasis interupsi eksternal menunjukkan tingkat presisi yang sangat tinggi pada frekuensi rendah hingga menengah. Pada frekuensi 1 kHz dan 10 kHz, ESP32 mampu mencacah seluruh pulsa masukan secara sempurna tanpa adanya selisih. Ketika frekuensi ditingkatkan menjadi 100 kHz dan 200 kHz, nilai frekuensi terukur masing-masing berada pada 99,993 kHz dan 199,5 kHz. Penyimpangan kecil ini disebabkan oleh penggunaan fungsi delay(1000) sebagai jendela waktu pengukuran yang memiliki jitter dan ketidakpastian pewaktuan akibat eksekusi tugas latar belakang pada sistem operasi real-time (FreeRTOS) ESP32.

Batas maksimum frekuensi yang dapat diukur secara andal tanpa kehilangan pulsa berada pada kisaran 200 kHz. Ketika frekuensi signal generator ditingkatkan melebihi batas ini hingga mencapai 456 kHz, frekuensi yang terukur oleh ESP32 turun drastis menjadi 385 kHz. Hal ini menunjukkan bahwa sistem telah mengalami fenomena lost pulses di mana pulsa masukan hilang dan tidak terdeteksi.

Fenomena hilangnya pembacaan pulsa pada frekuensi tinggi ini disebabkan oleh beberapa faktor teknis yang saling berkaitan. Pertama, terjadi interrupt overhead dan context switching, di mana setiap kali sinyal memicu transisi Rising Edge, CPU ESP32 harus menghentikan alur program utama, menyimpan status register, mengeksekusi ISR count_interup(), dan memulihkan kembali status register. Pada frekuensi 456 kHz, periode antar-pulsa sangat singkat ($\sim 2,19 \mu\text{s}$) yang mendekati atau lebih kecil dari total waktu overhead pemunculan interupsi dan eksekusi ISR, sehingga pulsa baru yang masuk saat CPU masih menyelesaikan siklus interupsi sebelumnya tidak

dapat dicatat. Ditambah lagi, keterbatasan fungsi delay(1000) yang terpengaruh oleh overhead instruksi pencetakan serial Serial.print() membuat jendela waktu tidak tepat 1,000 detik. Oleh karena itu, untuk pengukuran frekuensi tinggi di atas 200 kHz, pendekatan software interrupt ini idealnya digantikan oleh fitur hardware counter internal ESP32 seperti periferal Pulse Counter (PCNT).

5. KESIMPULAN

Berdasarkan seluruh rangkaian percobaan dan analisis data yang telah dilakukan pada Praktikum Modul 1, diperoleh beberapa kesimpulan sebagai berikut:

- 1) Modul mikrokontroler ESP32 memiliki alamat MAC unik d8:bc:38:f7:d3:54 yang tersimpan secara permanen pada memori eFuse OTP, serta memiliki tegangan kerja terukur Vcc sebesar 3,223 V yang stabil untuk mengoperasikan periferal I/O digital.
- 2) Penggunaan metode Direct Register Access (GPIO_OUT_REG) menghasilkan kecepatan toggling rate sinyal keluaran digital yang jauh lebih tinggi dan simetris (duty cycle $\sim 50,0\%$) dibandingkan metode pustaka standar Arduino (digitalWrite()). Pada frekuensi CPU maksimum 240 MHz, akses register menghasilkan frekuensi keluaran hingga 4,000 MHz, sedangkan pustaka standar Arduino hanya mencapai 1,299 MHz dengan duty cycle asimetris ($\sim 41,6\%$) akibat adanya overhead eksekusi instruksi dan pemanggilan tugas latar belakang FreeRTOS.
- 3) Metode Direct Register Access mampu meminimalisasi latensi pemrosesan I/O digital hingga sebesar 486,66 ns pada frekuensi 100 kHz (penurunan latensi lebih dari 56% dibanding pustaka Arduino sebesar 1,114 μs). Hal ini sekaligus memperluas bandwidth frekuensi sampling masukan maksimum dari 800 kHz pada pustaka Arduino menjadi 1 MHz pada metode akses register langsung.
- 4) Relay elektromekanik memiliki latensi pemunculan dalam orde milisekon akibat adanya penundaan magnetisasi kumparan dan inersia pergerakan mekanis lidah kontak. Waktu respon terukur osiloskop menunjukkan durasi 3,78 ms untuk transisi rising edge dan 3,50 ms untuk falling edge (terbaca 4 ms pada pembulatan perangkat

lunak), sehingga relay tidak cocok untuk aplikasi frekuensi tinggi seperti PWM.

- 5) Metode pencacahan pulsa berbasis external interrupt memberikan tingkat presisi pengukuran frekuensi yang sangat tinggi hingga frekuensi masukan 200 kHz. Pengukuran pada frekuensi di atas 200 kHz (seperti 456 kHz yang hanya terukur 385 kHz) mengalami fenomena lost pulses yang disebabkan oleh keterbatasan overhead pemicuan interupsi, durasi context switching CPU, serta jitter pewaktuan fungsi penundaan perangkat lunak.

DAFTAR PUSTAKA

- [1] ESPRESSIF, “ESP32 Series Datasheet Version 5.0 2.4 GHz Wi-Fi + Bluetooth,” Aug. 2025.
- [2] ESPRESSIF, “ESP32 Technical Reference Manual,” Aug. 2025.
- [3] AndrewR, “How Arduino digitalWrite Works - and Why AVR Is Faster,” crashbang_proto. Accessed: Sep. 26, 2026. [Online]. Available: <http://www.crash-bang.com/arduino-digital-io/>
- [4] J. W. Valvano, *Embedded Systems Real-Time Interfacing to ARM® Cortex(TM)-M Microcontrollers*. CreateSpace Independent Publishing Platform, 2011.
- [5] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design : The Hardware/Software Interface*. Amsterdam ; Boston: Elsevier/Morgan Kaufmann, 2020.
- [6] Q. Li and C. Yao, *Real-Time Concepts for Embedded Systems*. Boca Raton, FL: CRC Press, 2003.
- [7] E. Ashford and S. A. Seshia, *Introduction to Embedded Systems : A Cyber-Physical Systems Approach*. Cambridge, Ma: Mit Press, 2017.
- [8] C. W. de Silva, *Sensors and Actuators Engineering System Instrumentation, Second Edition*. Taylor & Francis, 2015.
- [9] A. S. Sedra, K. Carless Smith, T. C. Carusone, and V. Gaudet, *Microelectronic Circuits*. Oxford University Press, 2020.
- [10] R. Kamal, *Embedded Systems: Architecture, Programming and Design*. 2008.

Lampiran

Github Repo Praktikum :

<https://github.com/AqeelGhani/Modul1PraktikumSistemMikroprosesor>